

Université Ibn Zohr – Agadir
Faculté Polydisciplinaire de Ouarzazate
Département Informatique

Master : Intelligence Artificielle et Applications
Module : Intelligence Artificielle et Systèmes Multi-Agents

Systeme multi-agents pour la gestion des urgences hospitalières

Implémentation sous la plateforme JADE

Réalisé par :

Hamza Oukhacha
EL BLAIDI OTHMAN
ARCHAKI MOHAMED

Encadré par :

Dr. Imane Dbibih

Année universitaire 2026 – 2027

Résumé

Dans un service d'urgences, les patients arrivent en continu avec des niveaux de gravité très différents, et les docteurs n'ont pas tous la même charge de travail. Il faut donc décider vite quel patient va chez quel docteur. Dans ce projet, nous avons proposé une solution basée sur les systèmes multi-agents. Nous avons développé, avec la plateforme JADE, trois types d'agents : le Patient, l'Infirmier et le Docteur. Chaque docteur publie sa spécialité dans l'annuaire DF. L'infirmier fait le triage du patient (niveau d'urgence et spécialité), puis il lance une négociation avec les docteurs grâce au protocole Contract Net (CFP, PROPOSE, REFUSE, ACCEPT, REJECT, INFORM). Pour choisir le meilleur docteur, l'infirmier utilise une fonction de score multicritère qui combine le délai, la file d'attente et la spécialité ; le poids du délai augmente avec l'urgence. Nous avons aussi ajouté une interface graphique Swing et une sauvegarde des consultations dans des fichiers JSON. Les tests montrent que le système choisit le bon docteur même quand un autre docteur est aussi rapide mais hors spécialité, et qu'il gère bien les cas de saturation.

Mots-clés : systèmes multi-agents, JADE, protocole Contract Net, annuaire DF, triage hospitalier, décision multicritère.

Table des matières

Résumé	1
1 Introduction	5
2 Analyse et conception	7
2.1 Identification des agents	7
2.2 Diagramme d'interaction	8
2.3 Protocole de communication	9
2.4 Comportements de chaque agent	10
3 Implémentation	11
3.1 Architecture des conteneurs	11
3.2 Cycle de vie des agents	12
3.3 Publication et recherche de services (DF)	12
3.4 Le protocole Contract Net	13
3.5 Le triage	15
3.6 La fonction d'évaluation multicritère	15
3.7 Interface graphique et persistance	16
4 Résultats et expérimentations	18
4.1 Déploiement de la plateforme	18
4.2 Déroulement d'une négociation	19
4.3 Analyse d'un cas représentatif	20
4.4 Comportement en situation de saturation	20
5 Limites et perspectives	22
5.1 Limites de la solution actuelle	22
5.2 Perspectives d'amélioration	22
6 Conclusion	24

Table des figures

2.1	Diagramme d'interaction du système	8
4.1	Plateforme JADE : les quatre conteneurs déployés	18
4.2	Interface graphique et journal des messages ACL	19

Liste des tableaux

2.1	Rôles des agents du système	7
2.2	Messages ACL échangés	9
2.3	Comportements JADE utilisés par agent	10
4.1	Évaluation des propositions pour un cas de cardiologie ($u = 4$)	20

Chapitre 1

Introduction

La gestion est importante dans tous les domaines, mais elle devient critique dans le domaine hospitalier. Un service d'urgences reçoit en effet un flux continu de patients dont les degrés de gravité varient beaucoup. Il faut déterminer rapidement quelle situation est dangereuse et orienter chaque cas vers le praticien le plus adapté. De plus, les docteurs ne sont pas tous libres au même moment : certains ont une longue file d'attente, d'autres sont disponibles.

Ce type de problème se prête mal à une approche centralisée. En effet, chaque docteur est le seul à connaître sa propre file d'attente, et c'est à lui d'accepter ou de refuser une demande selon sa charge de travail. Le paradigme des systèmes multi-agents (SMA) est donc bien adapté : chaque acteur est représenté par un agent autonome, qui possède une information locale et qui peut négocier avec les autres. Cette architecture offre aussi une grande flexibilité : par exemple, on peut ajouter ou retirer un docteur pendant le fonctionnement, et les agents se découvrent entre eux grâce à l'annuaire DF.

Dans ce projet, nous avons développé un système multi-agents pour la gestion des consultations dans un service d'urgences, implémenté sous la plateforme JADE. Le système repose sur trois types d'agents :

- l'agent **Patient**, qui émet une demande de consultation ;
- l'agent **Infirmier**, qui fait le triage, négocie et décide ;
- l'agent **Docteur**, qui publie son service et fait des propositions.

Nous avons utilisé le protocole Contract Net pour la distribution des tâches : par exemple, le docteur fait une proposition s'il est disponible, sinon il refuse. C'est ce qui donne de la flexibilité au système. L'originalité de ce travail porte sur une fonction d'évaluation multicritère : l'infirmier ne choisit pas simplement le docteur le plus rapide, mais il combine le délai proposé, la taille de la file d'attente et la spécialité. De plus, le poids accordé au délai augmente avec le degré d'urgence du patient, ce qui permet à l'agent d'adapter ses critères au contexte.

Ce rapport s'organise comme suit. Le deuxième chapitre présente l'analyse et la conception du système : identification des agents, diagramme d'interaction et protocole de communication. Le troisième chapitre détaille l'implémentation sous la plateforme JADE, notamment la fonction d'évaluation multicritère. Le quatrième chapitre présente les résultats obtenus lors des expérimentations. Enfin, le dernier chapitre expose les limites de la solution actuelle ainsi que les perspectives d'amélioration.

Chapitre 2

Analyse et conception

2.1 Identification des agents

Pour construire le système, nous sommes partis du schéma vu en atelier : Consommateur, Acheteur et Vendeurs. Nous l'avons adapté au domaine de l'hôpital. Le patient joue le rôle du consommateur, l'infirmier joue le rôle de l'acheteur, et les docteurs jouent le rôle des vendeurs. Ce découpage est logique, car dans un vrai hôpital ce sont bien trois acteurs différents, avec des informations et des objectifs différents : le patient connaît ses symptômes, l'infirmier sait faire le triage, et le docteur connaît sa propre charge de travail.

TABLE 2.1 – Rôles des agents du système

Agent	Rôle principal	Conteneur
PatientAgent	Émettre une demande de consultation	Container-Patients
InfirmierAgent	Trier, négocier, décider	Container-Infirmier
DocteurAgent	Publier son service, proposer	Container-Docteurs

Chaque type d'agent est placé dans son propre conteneur. Le conteneur principal (Main-Container) contient les deux agents de la plateforme : l'AMS, qui gère les noms des agents, et le DF, qui joue le rôle de pages jaunes.

2.2 Diagramme d'interaction

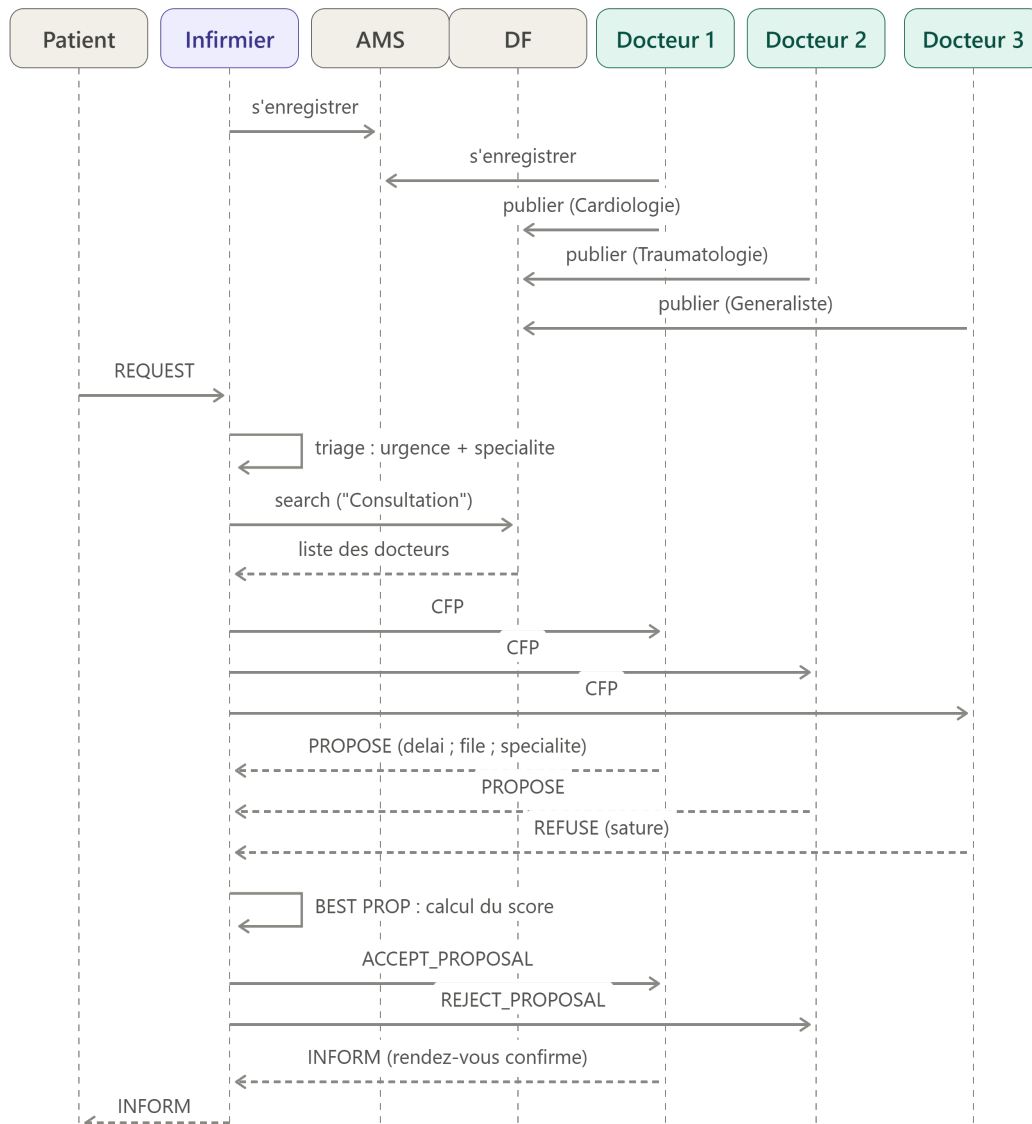


FIGURE 2.1 – Diagramme d'interaction du système

Comme le montre la figure 2.1, l'agent Infirmier et les agents Docteur s'enregistrent d'abord auprès de l'AMS, puis chaque docteur publie sa spécialité dans l'annuaire DF.

Le patient envoie une requête à l'agent Infirmier, qui reçoit la demande et détermine le degré d'urgence ainsi que la spécialité requise. L'agent Infirmier interroge ensuite l'annuaire DF afin d'identifier les docteurs disponibles, puis leur envoie un appel d'offres (CFP). Chaque docteur répond alors par une proposition (PROPOSE) ou un refus (REFUSE) selon sa charge de travail.

L'agent Infirmier évalue les propositions reçues, puis sélectionne la meilleure. Il envoie ensuite ACCEPT_PROPOSAL au docteur gagnant et REJECT_PROPOSAL

aux autres. Le docteur retenu confirme le rendez-vous (INFORM), puis l'infirmier transmet la confirmation au patient, ce qui clôture l'interaction.

2.3 Protocole de communication

Tous les messages utilisent la langue "Fr" et la même ontologie "Sante-Hopital". Le contenu est une simple chaîne de caractères où les champs sont séparés par un point-virgule. Le tableau 2.2 résume les messages échangés.

TABLE 2.2 – Messages ACL échangés

Émetteur → Ré- cepteur	Performatif	Contenu
Patient → Infirmier	REQUEST	symptôme
Infirmier → Docteurs	CFP	spécialité ; urgence
Docteur → Infirmier	PROPOSE	délai ; file ; spécialité
Docteur → Infirmier	REFUSE	motif de saturation
Infirmier → Gagnant	ACCEPT_PROPOSAL	Identifiant du patient
Infirmier → Autres	REJECT_PROPOSAL	Motif
Docteur → Infirmier	INFORM	docteur ; spécialité ; délai
Infirmier → Patient	INFORM / FAI- LURE	message final

2.4 Comportements de chaque agent

TABLE 2.3 – Comportements JADE utilisés par agent

Agent	Comportement	Tâche
Docteur	ParallelBehaviour	Exécute les trois comportements suivants en même temps
Docteur	CyclicBehaviour	Écoute les CFP et répond
Docteur	CyclicBehaviour	Traite ACCEPT/REJECT de l'infirmier
Docteur	TickerBehaviour	Termine une consultation toutes les 8 s
Infirmier	CyclicBehaviour	Écoute les demandes des patients
Infirmier	Behaviour (personnalisé)	Machine à états du Contract Net
Patient	WakerBehaviour	Attend 2 s avant d'envoyer la requête
Patient	CyclicBehaviour	Attend la réponse de l'infirmier

Le CyclicBehaviour attend passivement l'arrivée d'un message et se répète indéfiniment. Le TickerBehaviour s'exécute à intervalle régulier, indépendamment de l'arrivée d'un message. Le WakerBehaviour s'exécute une seule fois, après un délai déterminé.

Nous avons choisi ces comportements selon la nature de chaque tâche. Le docteur doit faire trois choses en même temps (répondre aux CFP, traiter les décisions et consulter ses patients), c'est pourquoi nous avons utilisé un ParallelBehaviour. Pour le patient, le WakerBehaviour de 2 secondes laisse le temps à l'infirmier et aux docteurs de s'inscrire dans le DF avant l'envoi de la requête. Enfin, la négociation de l'infirmier passe par plusieurs étapes, donc un Behaviour générique avec les méthodes `action()` et `done()` est le plus adapté.

Chapitre 3

Implémentation

3.1 Architecture des conteneurs

La classe `Runtime` gère le cycle de vie de la plateforme JADE et permet de créer les conteneurs. Sa méthode `instance()` renvoie toujours la même instance, car une seule plateforme JADE peut exister au sein d'une même JVM (patron Singleton).

La méthode `createMainContainer()` crée le conteneur principal, qui héberge automatiquement l'AMS et le DF. La méthode `createAgentContainer()` crée un conteneur secondaire, qui rejoint une plateforme existante via l'adresse `MAIN_HOST`. L'objet `ProfileImpl` sert à configurer chaque conteneur : par exemple `GUI = true` pour afficher la fenêtre RMA, ou `CONTAINER_NAME` pour donner un nom lisible au conteneur.

```
1 Runtime runtime = Runtime.instance();
2 ProfileImpl config = new ProfileImpl();
3 config.setParameter(ProfileImpl.MAIN_HOST, "localhost");
4 config.setParameter(ProfileImpl.CONTAINER_NAME, "Container-Docteurs");
5 AgentContainer conteneur = runtime.createAgentContainer(config);
6 conteneur.start();
```

Extrait de code 3.1 – Création d'un conteneur secondaire

Ensuite, chaque agent est créé dans son conteneur avec `createNewAgent()`, qui renvoie un `AgentController`. L'agent ne commence à vivre qu'après l'appel de `start()`.

```
1 Object[] arguments = new Object[]{
2     d.getSpecialite(),
3     String.valueOf(d.getDelaiBase()),
4     String.valueOf(d.getFileAttente()),
5     String.valueOf(d.getMaxFile())
6 };
7 AgentController controller = conteneur.createNewAgent(
8     d.getNom(), "agents.DocteurAgent", arguments);
9 controller.start();
```

Extrait de code 3.2 – Création et démarrage d'un agent Docteur

Nous avons aussi écrit une classe `LancerTout` qui démarre tout le système en une seule commande : le conteneur principal, puis les trois conteneurs secondaires et leurs agents.

3.2 Cycle de vie des agents

Chaque agent hérite de la classe `Agent` de JADE. La méthode `setup()` est appelée une seule fois, au démarrage de l'agent : c'est là que l'agent lit ses arguments, publie son service et ajoute ses comportements. La méthode `getArguments()` récupère le tableau d'objets passé au moment de la création. Par exemple, le docteur reçoit sa spécialité, son délai de base, sa file d'attente initiale et la taille maximale de sa file.

```

1 Object[] args = getArguments();
2 if (args != null && args.length >= 4) {
3     specialite = args[0].toString();
4     delaiBase = Integer.parseInt(args[1].toString());
5     fileAttente = Integer.parseInt(args[2].toString());
6     maxFile = Integer.parseInt(args[3].toString());
7 }

```

Extrait de code 3.3 – Lecture des arguments dans `setup()`

La méthode `takeDown()` est appelée juste avant la mort de l'agent. Nous l'utilisons pour retirer le service du DF avec `DFService.deregister()`, sinon l'infirmier pourrait envoyer un CFP à un docteur qui n'existe plus. Enfin, l'agent Patient appelle `doDelete()` dès qu'il reçoit la réponse finale : il quitte l'hôpital et son agent est détruit.

3.3 Publication et recherche de services (DF)

```

1 private void publierService() {
2     DFAgentDescription services = new DFAgentDescription();
3     services.setName(getAID());
4
5     ServiceDescription description = new ServiceDescription();
6     description.setType(TYPE_SERVICE);
7     description.setName(specialite);
8     services.addServices(description);
9
10    try {
11        DFService.register(this, services);
12        HopitalGUI.log(getLocalName(), "Service publie dans le DF
13                                : "
14                                + TYPE_SERVICE + " / " + specialite);
15    } catch (FIPAException e) {

```

```

15         e.printStackTrace();
16     }
17 }

```

Extrait de code 3.4 – Publication d'un service dans l'annuaire DF

La méthode `publierService()` enregistre le service d'un docteur dans l'annuaire DF, afin que les autres agents puissent le découvrir. Le type du service est "Consultation" pour tous les docteurs, et le nom du service contient la spécialité. De la même façon, l'infirmier publie un service de type "Triage", ce qui permet au patient de le trouver sans connaître son nom à l'avance.

Pour trouver les docteurs, l'infirmier crée un modèle de recherche avec seulement le type du service, puis il appelle `DFService.search()`. Le DF renvoie la liste de tous les agents qui correspondent.

```

1 DFAgentDescription modele = new DFAgentDescription();
2 ServiceDescription sd = new ServiceDescription();
3 sd.setType(DocteurAgent.TYPE_SERVICE);
4 modele.addServices(sd);
5 DFAgentDescription[] resultat = DFService.search(myAgent, modele);
6 AID[] aids = new AID[resultat.length];
7 for (int i = 0; i < resultat.length; i++) {
8     aids[i] = resultat[i].getName();
9 }

```

Extrait de code 3.5 – Recherche des docteurs dans le DF

Remarque : l'infirmier envoie le CFP à tous les docteurs, et pas seulement à ceux de la bonne spécialité. C'est un choix volontaire : si aucun spécialiste n'est libre, un autre docteur (par exemple un généraliste) peut quand même prendre le patient.

3.4 Le protocole Contract Net

La négociation est codée dans le comportement `NegocierConsultation` de l'infirmier. C'est une machine à états avec une variable `etape`. À chaque appel de `action()`, l'agent exécute l'étape courante, puis passe à la suivante. Le comportement se termine quand `done()` renvoie `true`, c'est-à-dire quand `etape == 4`.

```

1 public void action() {
2     switch (etape) {
3         case 0: envoyerCFP(); break;
4         case 1: collecterReponses(); break;
5         case 2: choisirMeilleure(); break;
6         case 3: attendreConfirmation(); break;
7     }
8 }

```

```

9 public boolean done() {
10     return etape == 4;
11 }

```

Extrait de code 3.6 – Machine à états de la négociation

Les quatre étapes sont les suivantes :

1. **Envoyer le CFP** : l’infirmier cherche les docteurs dans le DF et leur envoie un seul message CFP avec plusieurs destinataires.
2. **Collecter les réponses** : il reçoit les PROPOSE et les REFUSE. Il passe à l’étape suivante quand tous les docteurs ont répondu, ou quand le temps d’attente (3 secondes) est dépassé.
3. **Choisir la meilleure proposition** : il calcule un score pour chaque proposition, envoie ACCEPT_PROPOSAL au gagnant et REJECT_PROPOSAL aux autres.
4. **Attendre la confirmation** : il attend l’INFORM du docteur, enregistre la consultation, puis répond au patient.

Le point le plus important est le `conversationId`. Chaque patient reçoit un numéro de dossier (par exemple D0S-002), et ce numéro est utilisé comme identifiant de conversation dans le CFP. Grâce à la méthode `createReply()`, les réponses des docteurs gardent le même identifiant. L’infirmier filtre alors les messages avec un `MessageTemplate` :

```

1 tmplPropositions = MessageTemplate.and(
2     MessageTemplate.MatchConversationId(dossier),
3     MessageTemplate.or(
4         MessageTemplate.MatchPerformative(ACLMessage.PROPOSE),
5         MessageTemplate.MatchPerformative(ACLMessage.REFUSE)));

```

Extrait de code 3.7 – Filtrage des réponses par conversationId

Ainsi, si deux patients arrivent presque en même temps, l’infirmier lance deux négociations en parallèle et ne mélange jamais leurs messages. Nous avons aussi utilisé `block(200)` au lieu de `block()` dans l’étape de collecte : l’agent se réveille régulièrement pour vérifier si le délai d’attente est dépassé, même si aucun message n’arrive.

Du côté du docteur, la réponse au CFP dépend de sa charge :

```

1 if (fileAttente >= maxFile && urgence < 4) {
2     reponse.setPerformative(ACLMessage.REFUSE);
3     reponse.setContent("Sature : " + fileAttente + " patients en
4         attente");
5 } else {
6     int delaiPropose = delaiBase + fileAttente * 5;
7     reponse.setPerformative(ACLMessage.PROPOSE);

```

```

7     reponse.setContent(delaiPropose + ";" + fileAttente + ";" +
8         specialite);
    }

```

Extrait de code 3.8 – Réponse du docteur à un CFP

Le délai proposé augmente de 5 minutes pour chaque patient déjà en attente. Un docteur saturé refuse les cas légers, mais il accepte toujours de proposer pour les cas très urgents ou critiques (urgence 4 ou 5), car dans un hôpital on ne peut pas laisser un cas grave sans réponse.

3.5 Le triage

Avant la négociation, l’infirmier analyse le symptôme avec la classe **Triage**. Il cherche des mots-clés dans le texte pour donner un niveau d’urgence de 1 (faible) à 5 (critique), et pour trouver la spécialité requise. Par exemple, « douleur thoracique » donne une urgence de 4 (très urgent) et la spécialité Cardiologie, et « fracture du bras » donne une urgence de 3 (urgent) et la spécialité Traumatologie. C’est une méthode simple, mais elle suffit pour montrer le fonctionnement du système.

3.6 La fonction d’évaluation multicritère

Pour choisir le meilleur docteur, l’infirmier calcule un score pour chaque proposition. Le docteur qui a le score le plus petit gagne.

$$\text{score} = (1 + u) \times d + w_f \times f + \begin{cases} 0 & \text{si la spécialité correspond} \\ P & \text{sinon} \end{cases} \quad (3.1)$$

où u désigne le niveau d’urgence, d le délai proposé, f la taille de la file d’attente, w_f le poids de la charge et P la pénalité de spécialité. Dans notre implémentation, $w_f = 5$ et $P = 50$.

```

1 public static final double W_DELAI = 1.0;
2 public static final double W_FILE = 5.0;
3 public static final double PENALITE_SPECIALITE = 50.0;
4
5 public static double score(int delai, int fileAttente,
6                             boolean bonneSpecialite, int urgence) {
7     double w1 = W_DELAI * (1 + urgence);
8     double penalite = bonneSpecialite ? 0.0 : PENALITE_SPECIALITE;
9     return w1 * delai + W_FILE * fileAttente + penalite;
10 }

```

Extrait de code 3.9 – Classe Evalueur

Voici pourquoi nous avons choisi chaque terme :

- **Le délai** $(1 + u) \times d$: c'est le critère le plus important, car le patient veut être soigné vite. Le poids du délai dépend de l'urgence. Pour un cas faible ($u = 1$), une minute d'attente coûte 2 points ; pour un cas critique ($u = 5$), elle coûte 6 points. Donc plus le cas est grave, plus l'infirmier donne d'importance à la rapidité. Nous avons ajouté +1 pour que le délai compte toujours, même avec une urgence faible.
- **La file** $w_f \times f$: ce terme sert à équilibrer la charge entre les docteurs. Si deux docteurs proposent le même délai, on préfère celui qui a moins de patients, pour ne pas fatiguer toujours le même docteur.
- **La pénalité** P : un docteur hors spécialité peut quand même gagner, mais seulement s'il est beaucoup plus rapide. La pénalité n'est pas infinie, pour éviter qu'un patient reste sans docteur quand tous les spécialistes sont occupés.

Il y a un effet intéressant : quand l'urgence augmente, le poids du délai augmente, mais la pénalité reste fixe. Donc pour un cas critique, un docteur hors spécialité mais très rapide a plus de chances de gagner. C'est logique : pour un patient en danger, voir un docteur tout de suite est parfois plus important que voir le bon spécialiste plus tard.

3.7 Interface graphique et persistance

Nous avons développé une interface graphique avec Swing (classe `HopitalGUI`). Elle contient trois parties : un formulaire pour créer un nouveau patient (nom et symptôme), un tableau des docteurs enregistrés dans le DF avec leur file et leur état, et un journal qui affiche tous les messages ACL en temps réel.

Un point technique important : les agents JADE s'exécutent dans leurs propres threads, alors que Swing n'accepte les modifications que depuis son thread graphique (l'EDT). Si un agent modifie directement un tableau Swing, on peut avoir des erreurs ou un affichage bloqué. C'est pourquoi les agents ne touchent jamais Swing directement : ils appellent des méthodes statiques comme `HopitalGUI.log()`, qui utilisent `SwingUtilities.invokeLater()` pour envoyer la modification au thread graphique.

```

1 public static void log(final String source, final String message) {
2     SwingUtilities.invokeLater(new Runnable() {
3         public void run() {
4             // ajout de la ligne dans le journal
5         }
6     });
7 }
```

Extrait de code 3.10 – Mise à jour de l'interface depuis un agent

Pour la persistance, nous avons utilisé deux fichiers JSON. Le fichier `data/docteurs.json` contient la liste des docteurs ; il est lu au démarrage pour créer les agents Docteur. Donc pour ajouter un docteur, il suffit de modifier ce fichier, sans toucher au code. Le fichier `data/consultations.json` garde l'historique : après chaque consultation confirmée, l'infirmier y ajoute une ligne (dossier, patient, symptôme, urgence, docteur, délai, date). Pour ne pas dépendre d'une bibliothèque externe, nous avons écrit un petit lecteur JSON (`SimpleJson`) ; le projet fonctionne donc avec seulement `jade.jar`. La méthode d'écriture est `synchronized` pour éviter que deux négociations écrivent dans le fichier en même temps.

Chapitre 4

Résultats et expérimentations

4.1 Déploiement de la plateforme

Au lancement, la fenêtre RMA de JADE s'ouvre. La figure 4.1 montre les quatre conteneurs de la plateforme : le conteneur principal et les trois conteneurs que nous avons créés.

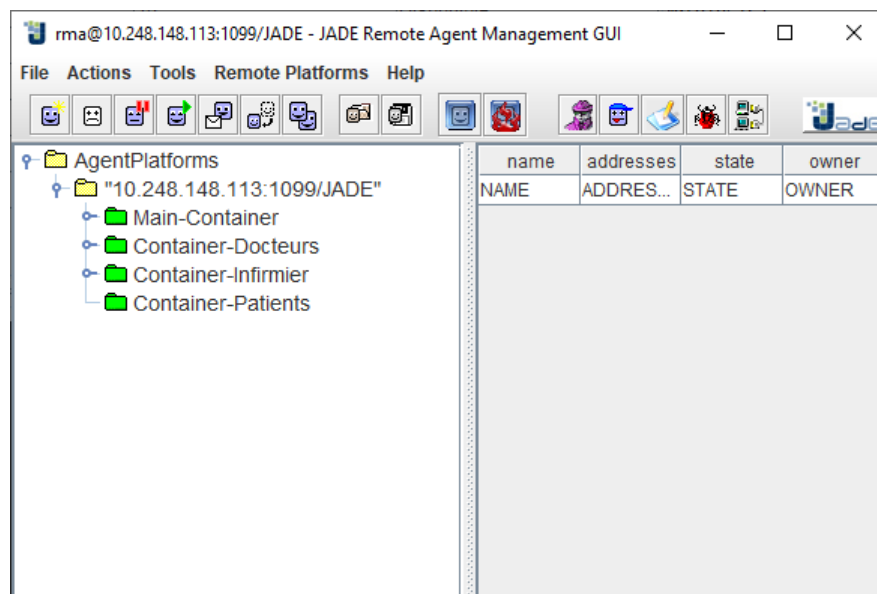


FIGURE 4.1 – Plateforme JADE : les quatre conteneurs déployés

Au démarrage, six docteurs sont créés à partir du fichier JSON : deux cardiologues (Dr_Alami et Dr_Bennis), un traumatologue (Dr_Douiri), un neurologue (Dr_Essaidi), un généraliste (Dr_Chafik) et un pneumologue (Dr_Fassi). Ils apparaissent tous dans le tableau de l'interface.

4.2 Déroulement d'une négociation

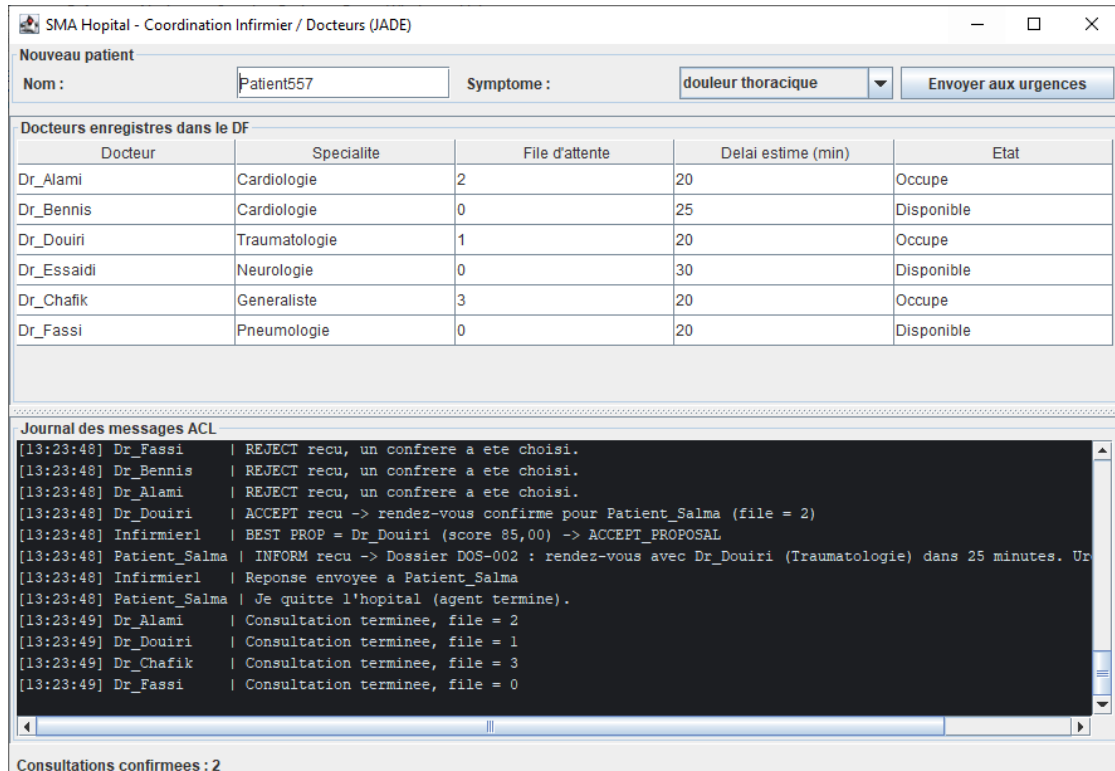


FIGURE 4.2 – Interface graphique et journal des messages ACL

La figure 4.2 montre la négociation pour la patiente Patient_Salma, qui arrive avec le symptôme « fracture du bras ». L’infirmier ouvre le dossier DOS-002 et le classe en urgence 3 (urgent), avec la spécialité Traumatologie. Il envoie ensuite un CFP aux six docteurs.

Après l’évaluation, Dr_Douiri (traumatologue) gagne avec un score de 85. Ce score se calcule facilement : délai de 20 minutes, file de 1 et bonne spécialité, donc $(1 + 3) \times 20 + 5 \times 1 + 0 = 85$. L’infirmier envoie ACCEPT_PROPOSAL à Dr_Douiri et REJECT_PROPOSAL aux autres (on voit dans le journal les lignes « REJECT reçu » de Dr_Fassi, Dr_Bennis et Dr_Alami). Dr_Douiri ajoute la patiente à sa file (file = 2), puis envoie INFORM. Enfin, la patiente reçoit le message final : rendez-vous avec Dr_Douiri dans 20 minutes, et son agent se termine. Le compteur en bas de la fenêtre affiche 2 consultations confirmées.

On voit aussi dans le journal le TickerBehaviour des docteurs : toutes les 8 secondes, chaque docteur termine une consultation et sa file diminue (« Consultation terminée »).

4.3 Analyse d'un cas représentatif

Pour montrer que la sélection est vraiment multicritère, nous prenons le premier patient, Patient_Youssef, qui arrive avec une « douleur thoracique ». Le triage donne une urgence $u = 4$ et la spécialité Cardiologie. Le poids du délai est donc $1 + 4 = 5$. Le tableau 4.1 donne les propositions reçues et les scores calculés.

TABLE 4.1 – Évaluation des propositions pour un cas de cardiologie ($u = 4$)

Docteur	Délai (min)	File	Spécialité	Score
Dr_Alami	20	2	Oui	110
Dr_Bennis	25	0	Oui	125
Dr_Douiri	20	1	Non	155
Dr_Fassi	25	1	Non	180
Dr_Chafik	25	4	Non	195
Dr_Essaidi	30	0	Non	200

Dr_Alami gagne avec un score de 110. Le cas de Dr_Douiri est le plus intéressant : il propose exactement le même délai que Dr_Alami (20 minutes) et il a même une file plus courte (1 patient au lieu de 2). S'il y avait seulement le critère de rapidité ou de charge, il aurait pu gagner. Mais il n'est pas cardiologue, donc il reçoit la pénalité de 50 points et il perd. Cela prouve que l'infirmier ne choisit pas seulement le plus rapide : il prend en compte les trois critères ensemble.

On remarque aussi que Dr_Bennis, qui a une file vide, perd contre Dr_Alami. Pour un cas très urgent, les 5 minutes de différence sur le délai coûtent 25 points, ce qui est plus important que les 2 patients en attente (10 points). L'urgence a donc bien un effet sur la décision.

4.4 Comportement en situation de saturation

Pour tester la saturation, nous avons envoyé plusieurs patients avec des symptômes légers, de façon rapide, depuis l'interface. Le généraliste Dr_Chafik, qui commence avec 4 patients et une file maximale de 6, se remplit rapidement. Quand sa file atteint 6, il répond REFUSE avec le message « Saturé : 6 patients en attente », et son état devient « Saturé » dans le tableau.

L'infirmier compte ce REFUSE comme une réponse, puis il continue la négociation avec les autres propositions. Le patient est donc envoyé chez un autre docteur. Si tous les docteurs refusent, l'infirmier envoie un message FAILURE au patient (« Tous les docteurs sont saturés ») au lieu de le laisser attendre sans réponse. Enfin, grâce au

TickerBehaviour, la file du docteur diminue avec le temps, et il redevient disponible pour les prochains patients. Pour les cas très urgents (urgence 4 ou 5), le docteur saturé fait quand même une proposition, mais avec un délai plus long, ce qui lui donne un mauvais score.

Chapitre 5

Limites et perspectives

5.1 Limites de la solution actuelle

- **Ontologie non typée** : le contenu des messages est une simple chaîne séparée par des points-virgules. Si un agent change l'ordre des champs, l'autre agent ne comprend plus le message. Une vraie ontologie JADE avec des classes Java serait plus sûre.
- **Délai d'attente fixe** : l'infirmier attend les propositions pendant 3 secondes, quel que soit le cas. Pour un cas critique, ce temps pourrait être trop long, et avec beaucoup de docteurs il pourrait être trop court.
- **Pas de gestion de la défaillance** : si le docteur gagnant tombe en panne après l'ACCEPT, le patient reçoit un FAILURE, mais l'infirmier ne relance pas une nouvelle négociation.
- **Triage simple** : le triage se base sur des mots-clés. Un symptôme mal écrit ou non prévu donne une urgence de 1 et la spécialité Généraliste.
- **Interface locale** : l'interface graphique est dans la même JVM que les agents. Elle ne peut pas afficher des agents lancés sur une autre machine.

5.2 Perspectives d'amélioration

- **Négociation itérative** : l'infirmier pourrait demander au docteur de proposer un meilleur délai, en plusieurs tours, au lieu d'un seul tour de propositions.
- **Réallocation pour les cas critiques** : quand un cas critique arrive, un docteur pourrait déplacer un patient léger de sa file vers un autre docteur pour libérer de la place.
- **Agent superviseur** : un agent pourrait surveiller toute la plateforme, détecter les docteurs qui ne répondent plus et relancer les négociations échouées.

- **Triage intelligent** : utiliser un modèle d'apprentissage automatique entraîné sur des données médicales pour mieux estimer l'urgence.
- **Réglage des poids** : les poids w_f et P ont été choisis à la main ; on pourrait les ajuster à partir de l'historique enregistré dans le fichier JSON.

Chapitre 6

Conclusion

Dans ce projet, nous avons travaillé sur un problème réel : orienter les patients d'un service d'urgences vers le bon docteur, en tenant compte de la gravité de leur cas et de la charge de chaque docteur. Nous avons montré qu'une approche multi-agents est bien adaptée, car chaque docteur garde sa propre information et décide lui-même s'il peut accepter un nouveau patient.

Nous avons construit, avec la plateforme JADE, un système complet avec trois types d'agents répartis sur plusieurs conteneurs. Les docteurs publient leur service dans l'annuaire DF, l'infirmier fait le triage puis négocie avec eux grâce au protocole Contract Net, et il choisit le meilleur docteur avec une fonction de score multicritère qui s'adapte à l'urgence. Une interface graphique et une sauvegarde en JSON complètent le système. Les expériences montrent que le choix ne dépend pas seulement de la rapidité, et que la saturation est bien gérée.

Ce projet nous a permis de mettre en pratique tout ce que nous avons vu dans les ateliers : les conteneurs, le cycle de vie des agents, les différents comportements, l'annuaire DF et les messages ACL. Nous avons aussi appris des points techniques importants, comme l'utilisation du `conversationId` pour séparer plusieurs négociations, et la communication sûre entre les threads des agents et l'interface Swing.